

Understanding *Microservices*

Before we write any code, let's understand **what** microservices are, **why** they exist, and **how** they differ from the traditional monolithic approach. Simple explanations first.

- ▶ No code in this lesson – concepts only
- ▶ Level: Beginner
- ▶ Next lesson: REST API

01 What Problem Are We Solving?

Imagine you're building an online shopping application. It needs to handle user accounts, product catalog, shopping cart, order processing, payment, and notifications. The question is: **how do you organize all this?**

There are two main ways to structure such an application:



Monolithic Architecture

Everything lives in **one single application**. All features are built, deployed, and scaled together as one unit.



Microservices Architecture

Each feature is a **separate small application**. They run independently and communicate with each other over the network.



Think of it like a restaurant

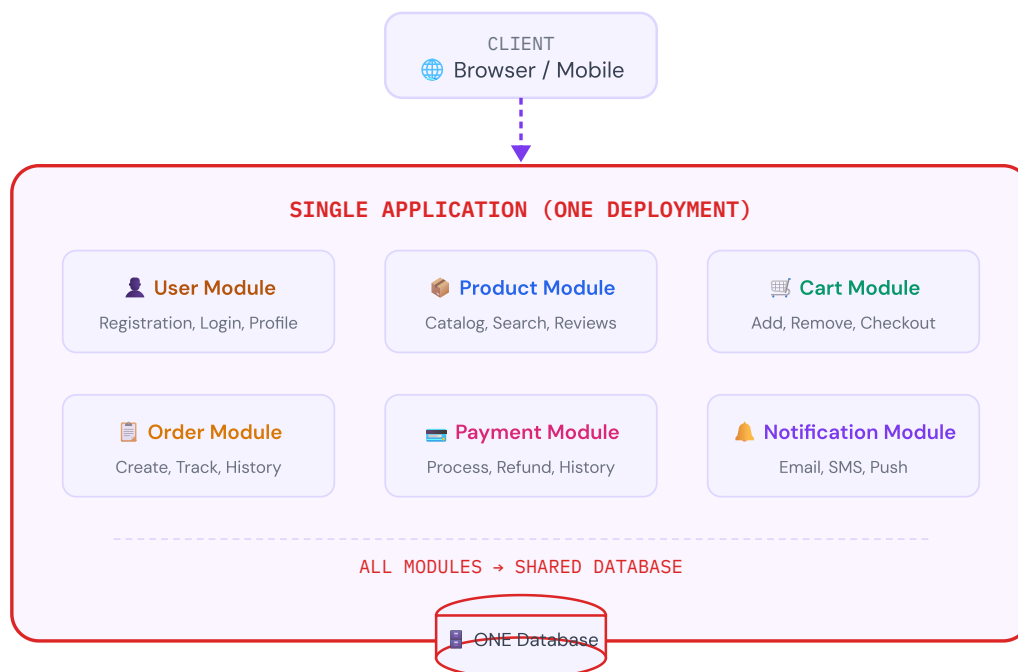
Monolith = One chef does everything: takes orders, cooks, plates, and serves. Fast to start, but if that chef gets sick, the whole restaurant stops.

Microservices = Separate stations: one for grilling, one for salads, one for desserts, one cashier. Each works independently. If the dessert station is slow, the rest of the restaurant keeps running.

02 The Monolith

In a monolithic architecture, all the components of your application are packaged and deployed as a **single unit**. All modules share the same codebase, the same database, and the same process.

MONOLITHIC ARCHITECTURE



How a Monolith Works

- All modules (User, Product, Cart, Order, Payment, Notification) are in **one codebase**
- They compile into **one JAR/WAR file** and deploy to **one server**
- All modules share the **same database**
- Modules call each other directly through **method calls** (fast, in-memory)
- To deploy a change in Payment, you must **redeploy the entire application**

✓ ADVANTAGES

- ✓ Simple to develop in the beginning
- ✓ Easy to test — everything in one place
- ✓ Simple deployment — one file to deploy
- ✓ Easy debugging — one process, one log
- ✓ No network latency between modules

✗ DISADVANTAGES

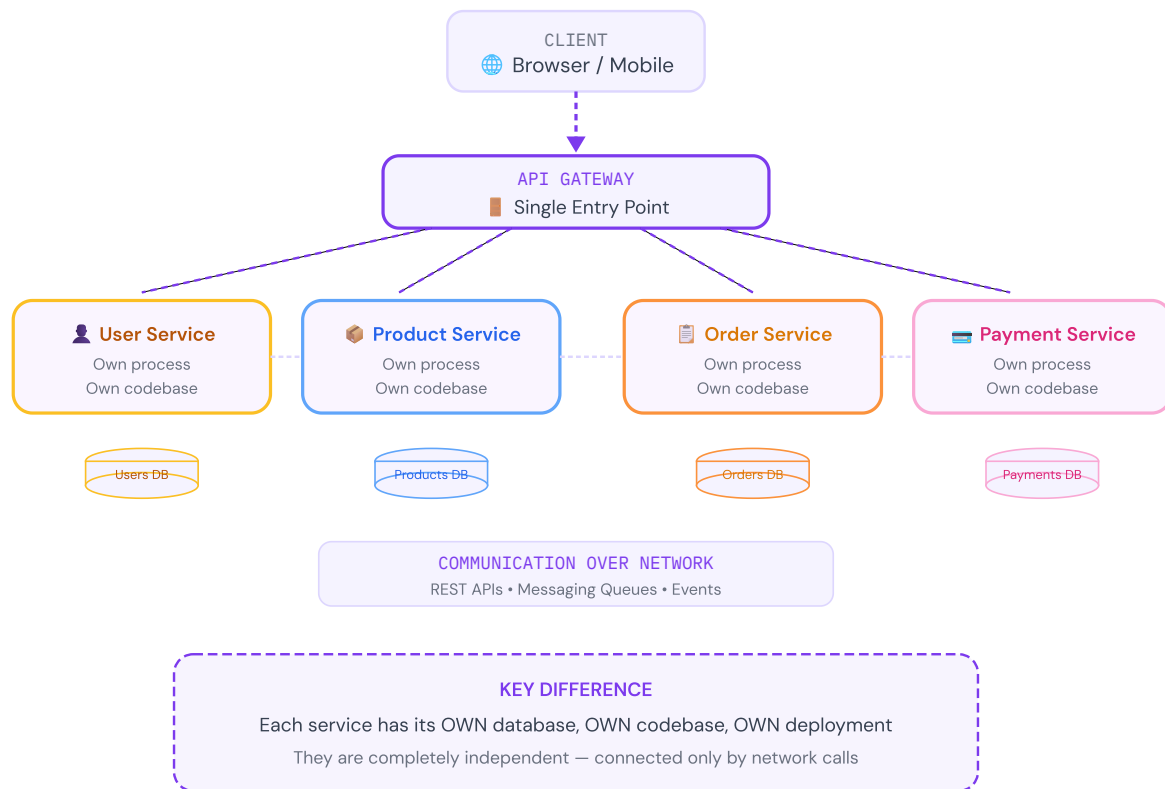
- ✗ Gets very complex as the app grows
- ✗ Small change = redeploy everything
- ✗ One bug can crash the entire system
- ✗ Cannot scale individual parts
- ✗ Team members block each other
- ✗ Stuck with one technology stack

The Monolith Problem: As your application grows (more features, more developers, more users), the monolith becomes harder to maintain, slower to deploy, and impossible to scale efficiently. This is often called the "Big Ball of Mud."

03 The Microservices Approach

Microservices solves the monolith's problems by **breaking the application into small, independent services**. Each service is a mini-application that handles one specific business function.

MICROSERVICES ARCHITECTURE



How Microservices Work

- Each service runs as a **separate application** (its own process, its own port)
- Each service has its **own database** — no sharing data directly
- Services communicate over the **network** using REST APIs or messages
- Each service can be **deployed independently** — update Payment without touching Orders
- Each service can be built with a **different technology** if needed
- Each service is owned by a **small, focused team**

✓ ADVANTAGES

- ✓ Independent deployment per service
- ✓ Scale only what needs scaling

✗ DISADVANTAGES

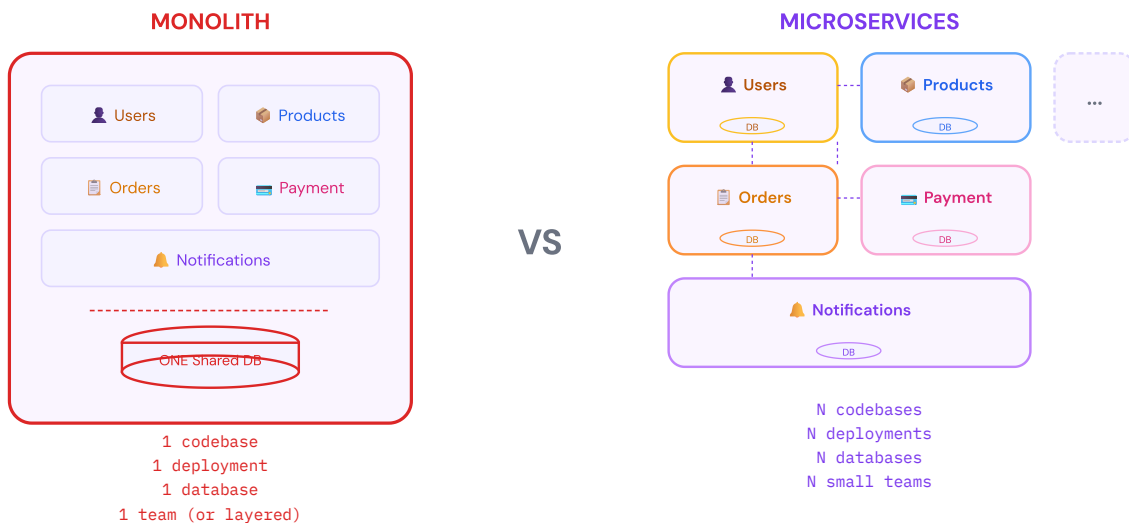
- ✗ Network calls are slower than method calls
- ✗ Harder to test across services

- ✓ Fault isolation — one crash ≠ total outage
- ✓ Teams work independently
- ✓ Technology freedom per service
- ✓ Easier to understand (small codebases)

- ✗ Distributed systems are complex
- ✗ Need extra infrastructure (gateway, registry)
- ✗ Data consistency is challenging
- ✗ Debugging across services is harder

04 Side-by-Side Comparison

MONOLITH VS MICROSERVICES – VISUAL COMPARISON



Comparison Table

ASPECT	MONOLITH	MICROSERVICES
Codebase	One repository, one project	Separate repository per service
Deployment	Deploy everything at once	Deploy each service independently

ASPECT	MONOLITH	MICROSERVICES
Database	One shared database	Each service has its own database
Scaling	Scale the whole application	Scale only the services that need it
Communication	Method calls (in-memory, fast)	Network calls — REST, gRPC, messages (slower)
Failure Impact	One bug can crash everything	One service fails, others keep running
Technology	One language, one framework	Different tech per service if needed
Team Structure	One large team or feature layers	Small team owns each service end-to-end
Complexity	Simple at first, painful later	Complex at first, manageable at scale
Testing	Easy — everything in one place	Harder — need integration & contract tests
Best For	Small apps, MVPs, small teams	Large apps, large teams, high scalability

05 When to Use Which?

Golden Rule: Start with a monolith. Move to microservices when you have a clear reason to — not because it sounds cool. Premature microservices is one of the most common architectural mistakes.

Stay with a Monolith when...

- Your team is small (under ~10 developers)
- You're building an MVP or a new product — you need to move fast
- The domain is not well understood yet (boundaries will change)
- You don't have the infrastructure skills for distributed systems
- The application is simple and doesn't need independent scaling

Consider Microservices when...

- The monolith has become too large and slow to deploy
- Multiple teams are stepping on each other's toes in the same codebase
- Different parts of the system need to scale independently (e.g., search gets 100x more traffic than checkout)
- You need different technologies for different parts
- Deployment speed matters — you want to release multiple times per day
- Your organization is structured around independent product teams



The Evolution Path

Most successful microservices systems started as monoliths. The typical path:

Monolith → **Modular Monolith** → **Extract Services Gradually**. Netflix, Amazon, and Uber all started as monoliths and migrated to microservices as they grew. Don't start with microservices on day one.

06 Key Concepts to Remember



Bounded Context

Each service handles one specific business domain. Order Service only knows about orders — it doesn't know how payments work internally.



Database per Service

Each service owns its data. If Order Service needs product info, it must **ask** Product Service — it cannot read the Products database directly.



API Gateway

A single entry point that routes client requests to the correct service. Clients don't need to know which service lives where.



Service Discovery

Services register themselves in a registry. When Service A needs to call Service B, it looks up the registry to find B's address.



Inter-Service Communication

Synchronous: Service A calls Service B and waits (REST API).

Asynchronous: Service A sends a message, Service B processes it later (message queue).



Fault Tolerance

Since any service can fail, you must plan for it: circuit breakers, retries, fallbacks, and timeouts. Assume failure will happen.

07 Real-World Examples

To make this concrete, here are well-known companies and how they use microservices:

COMPANY	STARTED AS	WHY THEY MIGRATED	# OF SERVICES
Netflix	Monolith	Needed to scale streaming globally; one outage took down everything	1,000+
Amazon	Monolith	Teams were blocked; couldn't deploy independently	Thousands
Uber	Monolith	Rapid feature growth; different scaling needs per feature	2,000+
Spotify	Monolith	Wanted autonomous "squads" that own full features end-to-end	800+

Important: These companies have thousands of engineers. Their problems are not your problems (yet). Use microservices to solve real scaling and team issues — not to copy what Netflix does.

Summary



What to take away from this lesson

1. A monolith is one big application — simple to start, hard to scale.
2. Microservices are many small applications — complex to start, easier to scale.
3. Each microservice has its own codebase, database, and deployment.
4. Services communicate over the network (REST or messaging).
5. Start with a monolith. Migrate to microservices when you outgrow it.
6. *Next lesson:* We'll learn about **REST APIs** — the foundation of how microservices communicate.

